Appendix A: Format Cover Page

UNIVERSITI TEKNOLOGI MARA
GROUP PROJECT

COURSE	:	**PROCESS CONTROL**
COURSE CODE	:	**ESE652**
SEMESTER	:	**2025/2026**

"I hereby declare that all submission is based on my original efforts, not plagiarised from any source(s), nor attempting to or communicating with any other person. I am fully aware that candidates who are suspected of violating this pledge are liable to be referred to the Disciplinary Committee of the College."

GROUP 5: EMPIRICAL MODELLING AND CONTROLLER TUNING OF FLOW RATE CONTROL IN A LIQUID FLOW SYSTEM.

NAME : **DAHLIA FATEHA BINTI AZIZI**
UiTM NO. : **2023262198**

NAME : **SYAZWAN AZRI BIN ZAIDI**
UiTM NO. : **2023867294**

NAME : **ANIS SUHANA BINTI MOHD AZMI**
UiTM NO. : **2023816852**

NAME : **AISYAH HANNAH BINTI MOHD ZAKI**
UiTM NO. : **2023660496**

NAME : **IMRAN BIN AHMAD SUHAIMI**
UiTM NO. : **2023660352**

Task	CO	PO	Level of Difficulty	WP, WKs, SDG	MARKS EARNED
1	CO2	PO2	C4	WK1 – WK4	/15
2	CO3	PO3	C4	WP, WK5	/25
				MARKS	/40

Table of Contents

1.0 THEORETICAL BACKGROUND 1

1.1 Introduction 1

1.2 Modelling 1

1.3 Proportional, Integral, Derivative (PID) Controllers 3

1.4 Controller Tuning 5

1.5 Performance Evaluation 6

2.0 PLANT DETAILS 9

2.1 P&I Diagram and Functional Block Diagram 9

2.2 Plant and Components 11

2.3 Challenges: Flow Process Control & Control Objective 13

3.0 EXPERIMENTAL PROCEDURE 14

3.1 Empirical Modelling 14

3.2 Simulation of PID Tuning Response 17

3.3 Real plant PID response 26

4.0 APPENDIX 28

1.0 THEORETICAL BACKGROUND

1.1 Introduction

Process control is a fundamental aspect of engineering that deals with regulating and managing various industrial processes. It involves the application of engineering principles to control and optimize the behaviour of a system. The goal of process control is to maintain desired output variables within a specified range, despite disturbances in the input or changes in operating conditions.

Process control consists of three main components which are transmitter, controllers, and final control element. Transmitter is a device that measure and provide information about the current state of the process (e.g. temperature, pressure, flow rate). Controllers on the other hand is a device or algorithm that analyse the sensor data and make decisions to adjust the process inputs. Lastly, final control elements are the mechanism that carry out the adjustment based on the signals from the controller such as valves and motors.

1.2 Modelling

Process modelling is the first and most critical step in control engineering. It involves building a mathematical model that represents the dynamic behaviour of a system when subjected to certain inputs. The most commonly used model in the industry is the First Order Plus Dead Time (FOPDT) model as it is able to capture the essential behaviour of complex systems in a simple and practical form. First Order Plus Dead Time (FOPDT) is a simple mathematical model that describes how industrial processes respond to changes over time.

The FOPDT model is fully describe by three main parameters which are process gain (Kp), time constant (τ), and dead time or time delay (θ). Process gain answers the question of how much the output changes for each unit change in input. Time constant describes how quickly the output responds, while dead time or time delay refers to how long it takes before the output starts to show signs of change after the input is changed.

The transfer function of FOPDT is written as follows:

$$G(s) = \frac{Kp}{\tau s + 1} e^{-\theta s}$$

where,

$Kp =$ Process Gain,

$\tau =$ Time constant,

$\theta =$ Dead Time / Time Delay.

The parameter Kp is calculated by the ratio of the final change in output (ΔPV) to the change in input (ΔMV). The equation for calculating this parameter through a step test is,

$$Kp = \frac{\Delta PV}{\Delta MV}$$

To obtain this model from experimental data, the Process Reaction Curve method is often used. There are several graphical techniques such as the Slope-Intercept method or the Sundaresan and Krishnaswamy method. In this graphical method, tangent lines are drawn at the inflection points to visually determine the values of θ and τ from the time response graph. Apart from the basic mathematical model, the difference between theoretical (physical) modelling and empirical modelling is very important. Empirical modelling is used when the physical information of the process is incomplete or too complex to be derived manually. The main advantage of empirical models is that they require less engineering effort because they are built directly from input-output (MV-PV) data taken during actual operation, but their validity is limited to the range of data used only.

In determining the FOPDT parameters empirically, the data collection steps must be carried out carefully. The process should be allowed to reach a steady state before the step test is initiated by abruptly changing the MV value. The Dead Time (θ) parameter is calculated as the time difference between the start of the step test and the first response of the output, while the Time Constant (τ) is usually determined by the 63.2% method, which is the time taken for the output to reach 63.2% of the final change.

For very complex systems with many time constants, higher-order transfer function approximation methods are required. Methods such as Taylor Series Expansion or Pade Approximations are used to represent the time delay $\left(e^{-\theta s}\right)$ in polynomial form so that stability analysis can be performed more easily. The use of Skogestad's Half Rule allows small time constants to be systematically distributed into dead time and main time constants, resulting in a simple model that still retains the original dynamic characteristics of the system.

1.3 Proportional, Integral, Derivative (PID) Controllers

PID controllers are the most widely used control algorithms in the process control industry due to their flexible structure and effectiveness in controlling a wide range of processes. The main objective of this controller is to minimize error, which is the difference between the desired setpoint value (r) and the measured process value (b), given by the equation of:

$$e = r - b$$

Proportional (P) control action provides an output that is directly proportional to the magnitude of the current error. Although this P action is fast, it has the major disadvantage of the existence of offset (a residual error that remains) where the system will not fully reach the setpoint after a load disturbance occurs.

The Proportional (P) mode can be expressed by:

$$p = K_c e_p + p_0$$

where,

$Kp =$ proportional gain between error and controller output (% per %),

$p_0 =$ controller output with no error (%)

Additionally, Proportional band is defined by the equation:

$$PB = \frac{100\%}{K_c}$$

The Integral (I) action is added to eliminate the offset. It works by accumulating the error over time and continuously changing the output until the error becomes zero. However, using too strong I action can cause the system to become unstable and produce large oscillations.

The equation for the Integral (I) action is:

$$p_i(t) = K_i \int_0^t e_p \, dt + p(0)$$

Derivative (D) action responds to the rate of change of the error. It is predictive by providing corrective action before the error becomes large, thus helping to dampen oscillations and speed up the system response. However, D mode is very sensitive to noise in the measurement signal and is rarely used alone.

The equation for the Derivative (I) action is:

$$p_d = K_d \frac{d}{dt} e_p$$

When these three modes are combined, they form a complete PID controller (Three-Mode Controller). This combination provides the advantages of fast Proportional (P) action, Integral (I) action that eliminates residual errors, and Derivative (D) action that stabilizes oscillations. The complete equation for a PID controller in the time domain is as follows:

$$p = K_c e_p + K_i \int_0^t e_p \, dt + K_d \frac{d}{dt} e_p + p(0)$$

where,

$$K_i = K_c \frac{1}{T_i}$$

$$K_d = K_c T_d$$

Therefore,

$$p = K_c e_p + K_c \frac{1}{T_i} \int_0^t e_p \, dt + K_c T_d \frac{d}{dt} e_p + p(0)$$

1.4 Controller Tuning

Controller tuning is the process of determining the best parameter values for Proportional Gain (Kc), Integral Time (Ti), and Derivative Time (Td) to achieve optimal system performance. These values are highly dependent on the unique characteristics of each process. Improper tuning can cause the system to respond too slowly or become unstable, damaging industrial equipment.

There are two main categories of PID tuning methods which are closed-loop tuning and open-loop tuning. For closed-loop tuning, the process is tuned while operating in closed-loop state. Meanwhile, open-loop tuning method is done based on information obtained from the process reaction curve (step test).

Among the open-loop methods that are often used are the Ziegler-Nichols, AMIGO, and Cohen-Coon method. Based on the FOPDT model identified from the reaction curve, tuning parameters, Kc, Ti, and Td for each Ziegler-Nichols, AMIGO and Cohen-Coon tuning methods can be calculated using the formulas provided in Table 1 below.

Table 1: PID Tuning formulas

PID Type	**Kc**	**Ti**	**Td**
Ziegler-Nichols (ZN)			
P	$\frac{T}{L}$	∞	0
PI	$0.9\frac{T}{L}$	$\frac{L}{0.3}$	0
PID	$1.2\frac{T}{L}$	$2L$	$0.5L$
AMIGO			
PI	$\frac{0.15}{Kp} + \left(0.35 - \frac{LT}{(L+T)^2}\right)\frac{T}{KpL}$	$0.35L + \frac{13LT^2}{T^2 + 12LT + 7L^2}$	0
PID	$\frac{1}{Kp}\left(0.2 + 0.45\frac{T}{L}\right)$	$\frac{0.4L + 0.8T}{L + 0.1T}L$	$\frac{0.5LT}{0.3L + T}$

Cohen Coon (CC)			
P	$\frac{3T+L}{3LKp}$	∞	0
PI	$\frac{10.8T+L}{12LKp}$	$\frac{L(30T+3L)}{9T+20L}$	0
PID	$\frac{16T+3L}{12LKp}$	$\frac{L(32T+6L)}{13T+8L}$	$\frac{4LT}{11T+2L}$

1.5 Performance Evaluation

Performance evaluation is the final step to ensure that the designed and tuned control system meets the specified operational objectives. The effectiveness of the system is evaluated based on three main criteria which are Step Test, Setpoint Tracking and Disturbance Rejection. Setpoint Test is a measure on how the system response to a step input. Setpoint tracking measures how well the system can follow changes in target values, while disturbance rejection tests the system's ability to return to the setpoint when there are unexpected external factors.

System performance is measured using dynamic response criteria that can be visually observed on the time response graph, which includes:

1. Rise Time, (Tr): The time it takes for the output to reach the setpoint value for the first time.
2. Settling Time, (Ts): The time it takes for the output to remain within the allowable error range.
3. Percentage Overshoot, (%OS): The maximum rate at which the output exceeds the setpoint value.
4. Steady State Error, (ess): The constant difference between the final output and the setpoint value.

In addition to graphical criteria, analytical evaluation using Time Integral Criteria provides a more comprehensive picture of the overall system response. This criterion calculates the accumulated error over the test period. The three commonly used criteria are ISE, IAE, and ITAE.

Integral of Squared Error (ISE) criterion emphasizes large errors because the error value is squared, making it suitable if we want to reduce excessive acceleration. Integral of Absolute Error (IAE) provides a balanced evaluation between initial and final errors, suitable for simulation studies. Meanwhile, Integral of Time-weighted Absolute Error (ITAE) gives a larger penalty to errors that occur late, thus prioritizing a system that stabilizes quickly.

The mathematical equation for the error integral criterion is as follows:

$$ISE = \int_0^\infty e^2\,(t)dt$$

$$IAE = \int_0^\infty |e(t)|\,dt$$

$$ITAE = \int_0^\infty t|e(t)|\,dt$$

The choice between ISE, IAE, or ITAE depends on the specific application requirements. For example, in a chemical reactor where long temperature changes can damage the product, the ITAE criterion may be preferred to ensure that the temperature returns to a stable state quickly. Overall, a better control system is one that has the smallest integral error value for the specified objective.

Furthermore, excessively high ramping (%OS) can not only affect product quality in the chemical industry, but can also cause high mechanical stress on valves and pumps. Visual assessment of the step response graph allows engineers to identify whether the system is under damped (oscillating) or over damped (slow) in order to make further adjustments.

ISE is suitable when there is a need to emphasize the minimization of larger errors and when controlling the variance in the system output is a priority. The IAE criterion is very popular in simulation studies because it calculates the total area under the error curve regardless of the direction of the error. While the ITAE criterion

is considered the most challenging because it forces the controller to stabilize the system as quickly as possible to avoid an error penalty that increases with time. In practice, the controller that produces the smallest error integral value in the setpoint tracking and disturbance rejection tests is considered to have the most superior performance.

2.0 PLANT DETAILS

2.1 P&I Diagram and Functional Block Diagram

P&I Diagram

Piping and Instrumentation Diagrams (P&I Diagrams) are detailed schematic representations of the mechanical and control instrumentation within any process system. It serves as a critical communication tool among engineers and operators, enabling comprehensive understanding of the process flow and control mechanisms implemented within a facility.

Below is the P&I diagram for the assigned flow control process plant. The full P&I diagram of the actual plant is attached in the Appendix.

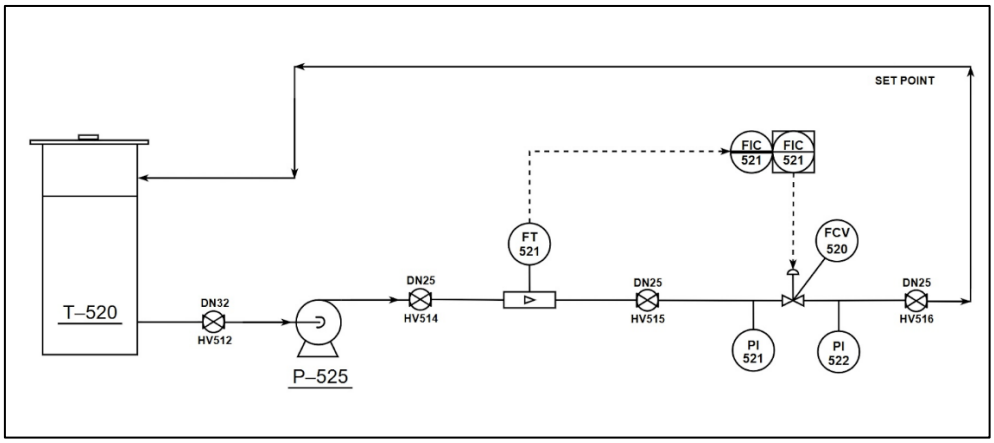


Figure 2.1: P&I Diagram of Flow Control Process Plant.

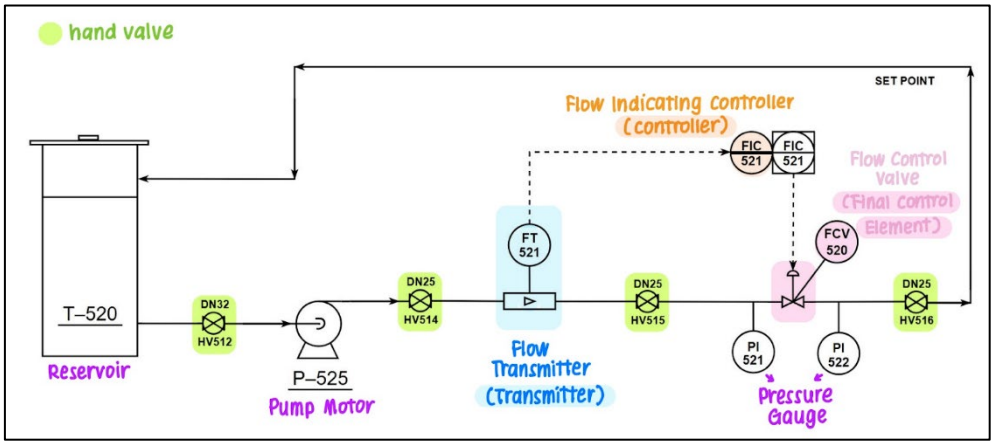


Figure 2.2: Annotated P&I Diagram of Flow Control Process Plant.

Functional Block Diagram of One Control Loop

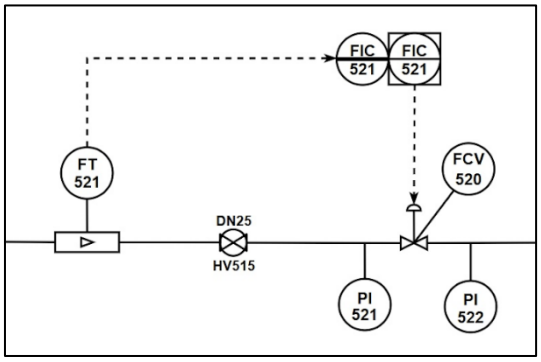


Figure 2.3: Cropped section of the control loop P&ID diagram.

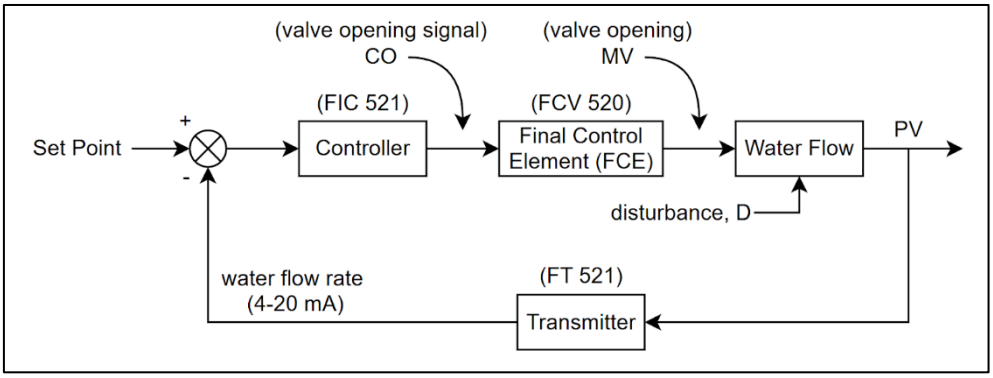


Figure 2.4: Functional Block Diagram of Flow Control Process Plant.

Figure 2.3 shows the subsystem of interest from the P&ID diagram. From the subsystem, a functional block diagram can be created to better visualize the process flow and each of the component's role within the subsystem. Figure 2.4 above shows the close loop model of the system which consist of components such as FIC521 Controller, FCV520 Final Control Element (FCE) and FT521 Transmitter.

2.2 Plant and Components

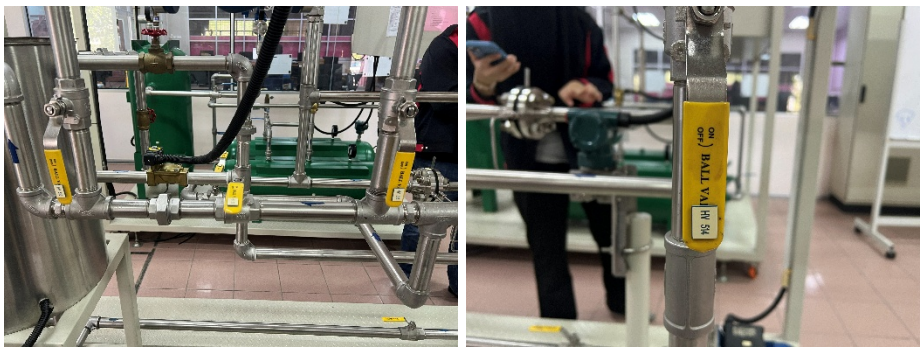


Figure 2.5: Real-life photo of Hand Valves.

The hand valve in this process is used to redirect or block water flow to a specific area. It must be correctly positioned to ensure proper isolation of the subsystem, allowing the process to operate correctly and removing the possibilities of interference from other parts of the plant. Ensuring the hand valve is in the correct position is an important first step before the controller is activated, as it helps inn reducing the risk of hazards or damage to the equipment, and avoiding system malfunctions.

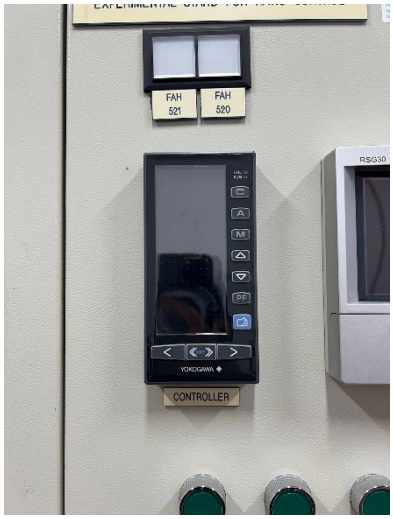


Figure 2.6: Real-life photo of Controller (FIC 521).

FIC521 is the Flow Input Controller of the flow control process plant subsystem. Its primary function is to regulate the process flow by adjusting the Valve Opening based on the desired set point. In manual mode, FIC521 allows direct control of the manipulated variable for an open-loop operation of the system. In automatic mode, the controller performs closed-loop control based on the set point and controlling it

through PID control. The controller output is a valve opening signal, which is sent to the FCV520 final control element to regulate the process flow accordingly.



Figure 2.7: Real-life photo of Final Control Element, FCE (FCV 520).

FCV520 is the Flow Control Valve that acts as the Final Control Element (FCE) of the system. It is the component that receives the controller output, which in this case is the valve opening signal, and responds accordingly to control the valve opening (MV). The change in MV will then causes the water flow rate (PV), to increase or decrease accordingly.



Figure 2.8: Real-life photo of Transmitter (FT 521).

FT521 is a Flow Transmitter that functions as the sensing element for the PV. It measures the water flow rate in the system and converts this measurement into a standardized electrical current signal (4–20 mA), which is transmitted to the controller. This feedback signal is used in the closed-loop control system to continuously calculate the error between the set point and the actual measured water flow rate.

2.3 Challenges: Flow Process Control & Control Objective

In real-world applications, water flow control is widely used within industries such as chemical processing, power generation, water treatment plants, and manufacturing systems, where in these applications, maintaining the desired flow rate is essential for proper mixing of substance, heat transfer and reaction rate. Poor flow control may result in unstable operation and overall inefficiencies, thus it is very important that flow control within a system is controlled effectively. Controlling water flow effectively is difficult as the process is commonly exposed to challenges such as pressure fluctuations, valve nonlinearities, and varying operation and environmental conditions.

The primary control objective in this flow control application is to achieve minimum settling time (Ts) so that the system can reach the desired water flow rate quickly and reliably following a set point change. In practical plant operation, rapid stabilization of flow is important to prevent prolonged deviations that may affect processes such as mixing, heat transfer, or production process. A control system with a minimal settling time will contributes to improving overall process efficiency, thus making it a key performance requirement for industrial water flow control systems.

3.0 EXPERIMENTAL PROCEDURE

3.1 Empirical Modelling

In making the empirical modelling, the process plant data is obtained through data collection process. The data collection process is done on the assigned process plant which is Flow Control plant to achieve the reaction curve. The data collected is as follows:

	A	B	C	D	E
1	no	time	mv	pv	
2	0	0	50	24.5	
3	1	0.2	50	24.5	
4	2	0.4	50	24.5	
5	3	0.6	50	24.5	
6	4	0.8	50	24.6	
7	5	1	50.1	24.6	
8	6	1.2	50.1	24.7	
9	7	1.4	50.2	24.8	
10	8	1.6	50.3	24.8	
11	9	1.8	50.4	24.9	
12	10	2	50.6	24.9	

Figure 3.1: Snippet of collected data.

FOPDT 1

From the process reaction curve above, the FOPDT can be obtained. In determining the model parameters gain (Kp), time constant (τ), and time delay (θ), Sundaresan method is used to fit the FOPDT model.

$$Kp = \frac{\Delta PV}{\Delta MV} = \frac{33.6 - 24.5}{60 - 50} = \frac{9.1}{10} = 0.91$$

Using Sundaresan method,

$t_{step} = 0, \quad t_1\, at\ 35.3\%, \quad t_2\, at\ 85.3\%,$

$$t_1 = \left(\frac{35.3}{100} \times 9.1\right) + 24.5 = 27.71$$

$$\therefore\ at \approx 27.7, \qquad t_1 = 6.6s$$

$$t_2 = \left(\frac{85.3}{100} \times 9.1\right) + 24.5 = 32.26$$

$$\therefore\ at \approx 32.25, \qquad t_2 = 9.5s$$

Time constant, τ is defined as,

$$\tau = 0.67(t_2 - t_1)$$

$$\therefore\ \tau = 0.67(9.5 - 6.6) = 1.943$$

Time delay, θ is defined as,

$$\theta = 1.3t_1 - 0.29t_2$$

$$\therefore\ \theta = 1.3(6.6) - 0.29(9.5) = 11.335$$

The resulting FOPDT transfer function can be expressed as,

$$G(s) = \frac{0.91}{1.943s + 1} e^{-11.335s}$$

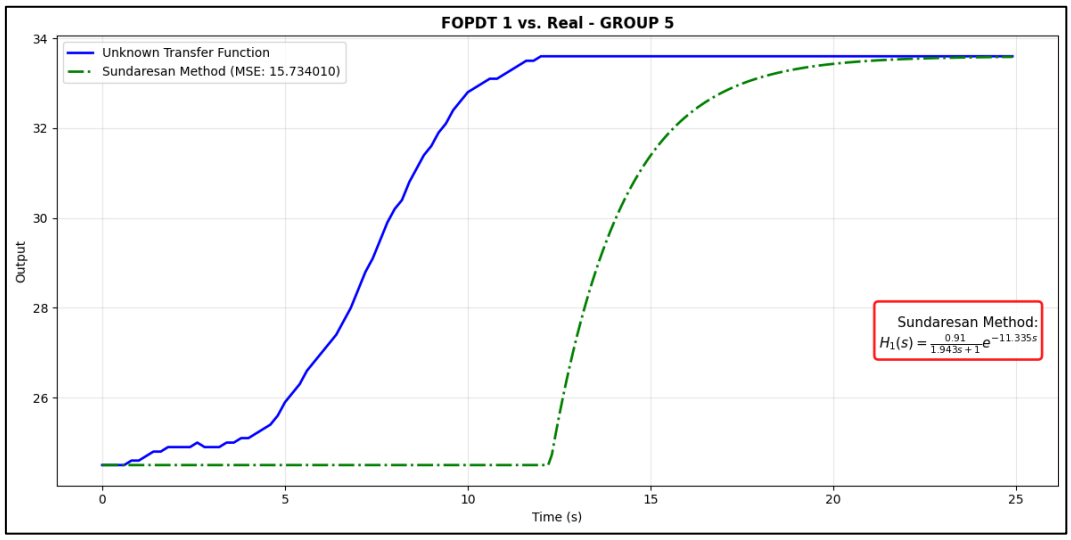


Figure 3.2: FOPDT 1 vs Real Plant Comparison.

FOPDT 2

```
# ==============================
# fopdt2 Method FOPDT
# ==============================
num_fopdt2 = [0.91]
den_fopdt2 = [2.5, 1]
sys_fopdt2 = signal.TransferFunction(num_fopdt2, den_fopdt2)
_, y_fopdt2, _ = signal.lsim(sys_fopdt2, U=u, T=t)

delay_fopdt2 = 4.3
delay_samples_fopdt2 = int(delay_fopdt2 / dt)

y_fopdt2_delayed = np.zeros_like(t)

if delay_samples_fopdt2 == 0:
    y_fopdt2_delayed = y_fopdt2.copy()
elif delay_samples_fopdt2 < len(t):
    y_fopdt2_delayed[delay_samples_fopdt2:] = y_fopdt2[:-delay_samples_fopdt2]

y_fopdt2_delayed += 24.5

# ==============================
# Sundaresan Method FOPDT
# ==============================
num_sundaresan = [0.91]
den_sundaresan = [1.943, 1]
sys_sundaresan = signal.TransferFunction(num_sundaresan, den_sundaresan)
_, y_sundaresan, _ = signal.lsim(sys_sundaresan, U=u, T=t)

delay_sundaresan = 11.335
delay_samples_sundaresan = int(delay_sundaresan / dt)

y_sundaresan_delayed = np.zeros_like(t)
```

Figure 3.3: Python coding snippets for FOPDT comparison.

In order to get a achieve a better approximation of the FOPDT, a python script was created in order to visualize and calculate the FOPDT accuracy against the real plant behaviour. Figure 3.3 above shows a snippet of the code used to read the value of plant readings from an excel sheet and visualize it against the FOPDT approximation.

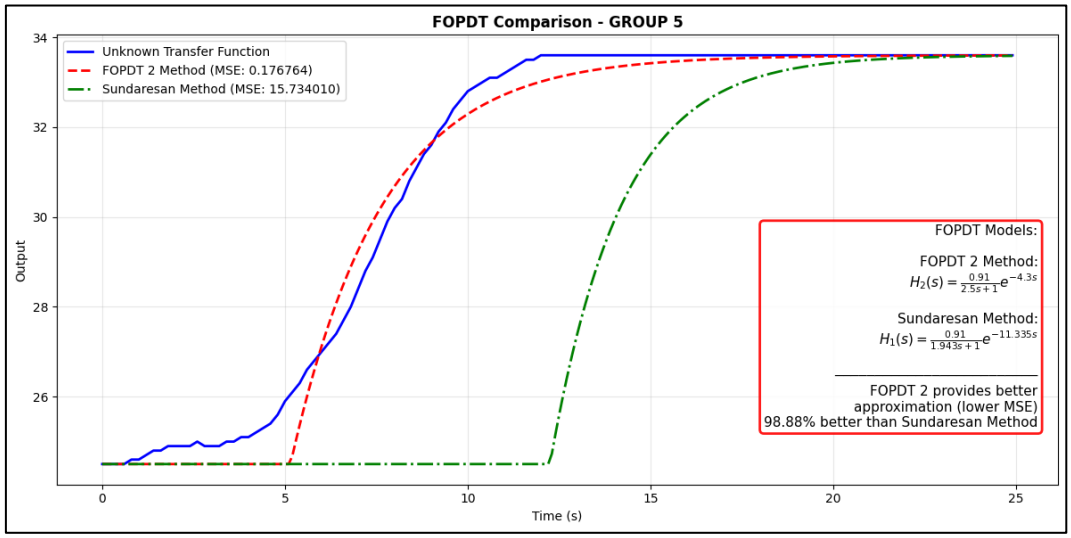


Figure 3.4: FOPDT 1 vs FOPDT 2 Against Real Plant Comparison.

Figure 3.4 shows the fine tuning of FOPDT 1 to further improve its accuracy. The decrease of time delay shifted FOPDT 1 to the left side, closer to the original signal. This modification to the transfer function decreased the MSE from 15.734 to 0.177 which was a 98.88% improvement from the original transfer function derived using Sundaresan Method.

3.2 Simulation of PID Tuning Response

	FOPDT 1			FOPDT 2	
	K	0.91		K	0.91
time delay	theta / L	11.335		theta / L	4.3
time const.	tau / T	1.943		tau / T	2.5

PID CALCULATION

ZN			
PID type	Kc	Ti	Td
P	0.5814	inf	0
PI	0.5233	14.3333	0
PID	0.6977	8.6000	2.1500

AMIGO			
PID type	Kc	Ti	Td
PI	0.2399	20.2612	0
PID	0.5073	3.5156	1.4182

COHEN-COON			
PID type	Kc	Ti	Td
P	1.0052	inf	0
PI	0.6666	3.4836	0
PID	1.1266	6.8003	1.1911

ZN - P, PI, PID			
PID type	KP value	KI value	KD value
P	0.5814	0	0
PI	0.5233	0.0365	0
PID	0.6977	0.0811	1.5000

AMIGO - PI, PID			
PID type	KP value	KI value	KD value
PI	0.2399	0.0118	0
PID	0.5073	0.1443	0.7194

CC - P, PI, PID			
PID type	KP value	KI value	KD value
P	1.0052	0	0
PI	0.6666	0.1913	0
PID	1.1266	0.1657	1.3419

Figure 3.5: Excel sheet for calculating PID parameters

Figure 3.5 above shows the excel sheet used to calculate every parameter of the ZN, AMIGO and Cohen-Coon PID Tuning based on FOPDT 2. Excel is used to make calculations easier when one or more of the FOPDT value is changed.

Tuning Response Evaluation

1. Step Test

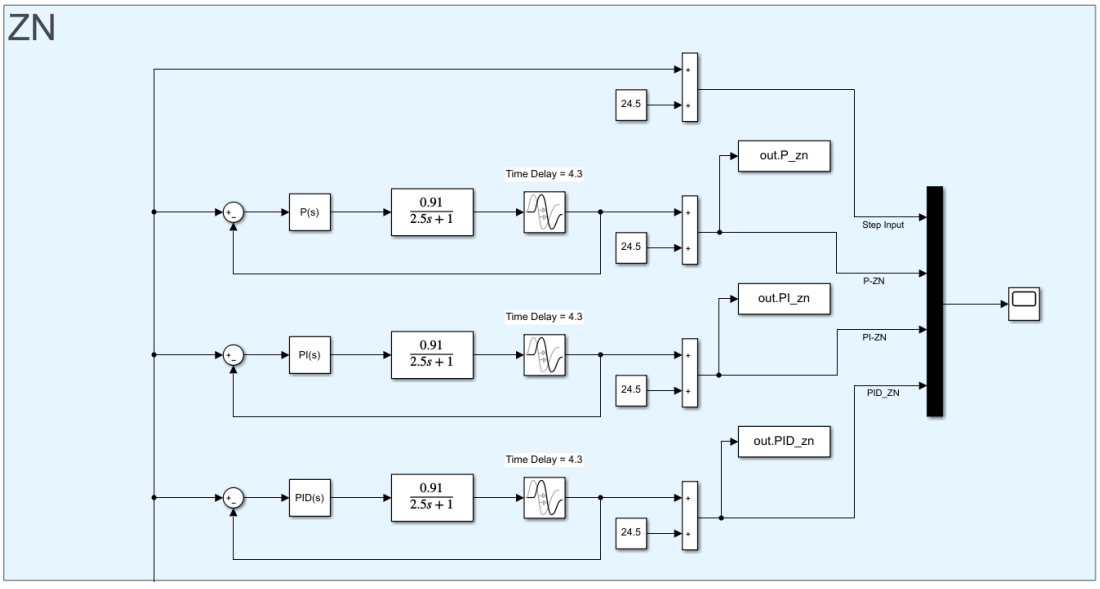
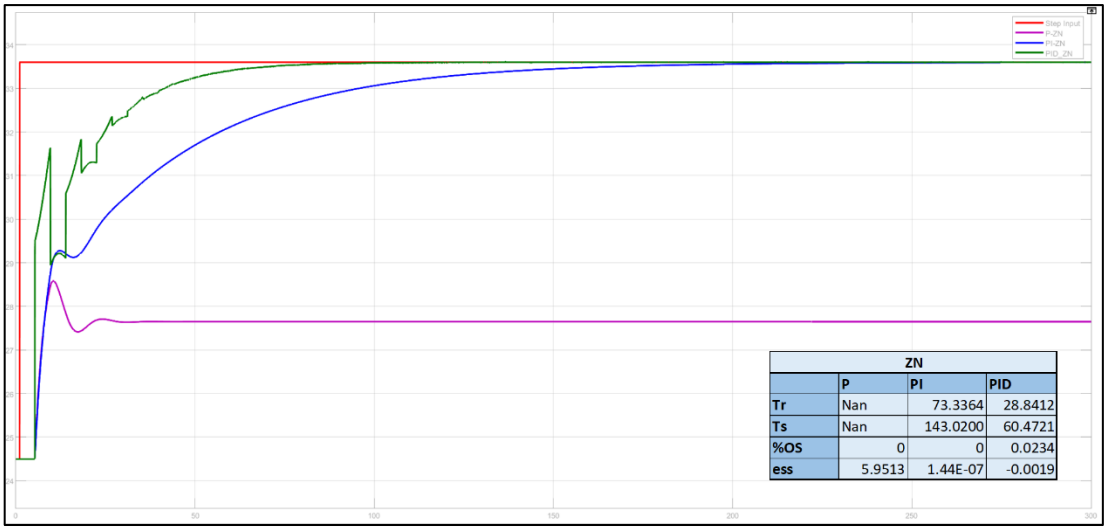


Figure 3.6: MATLAB Simulink diagram for ZN (P, PI, PID) Step Test.



ZN			
	P	PI	PID
Tr	Nan	73.3364	28.8412
Ts	Nan	143.0200	60.4721
%OS	0	0	0.0234
ess	5.9513	1.44E-07	-0.0019

Figure 3.7: Output Response & Dynamic Response for ZN (P, PI, PID) Step Test.

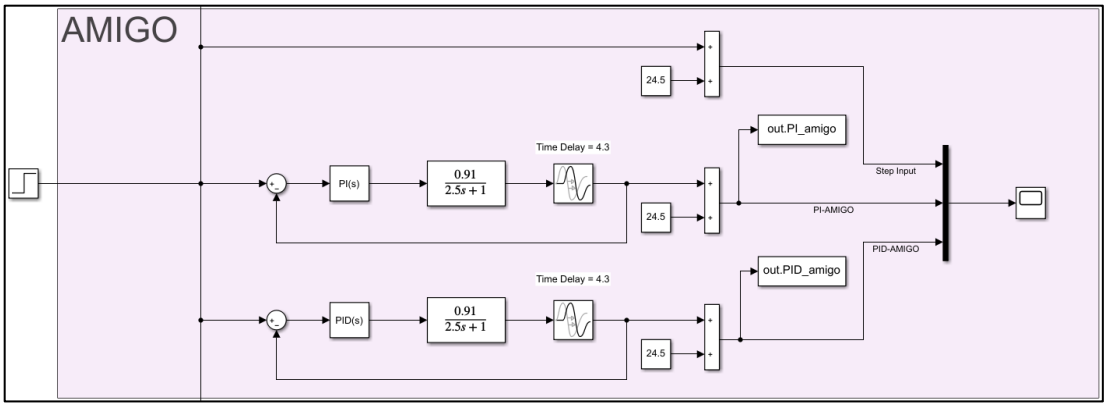
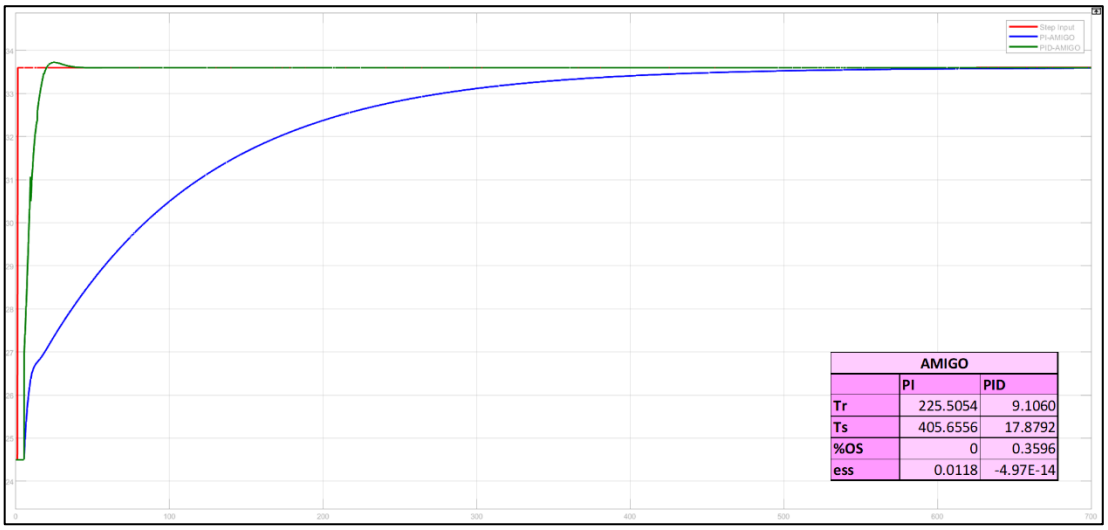


Figure 3.8: MATLAB Simulink diagram for AMIGO (PI, PID) Step Test.



AMIGO		
	PI	PID
Tr	225.5054	9.1060
Ts	405.6556	17.8792
%OS	0	0.3596
ess	0.0118	-4.97E-14

Figure 3.9: Output Response & Dynamic Response for AMIGO (PI, PID) Step Test.

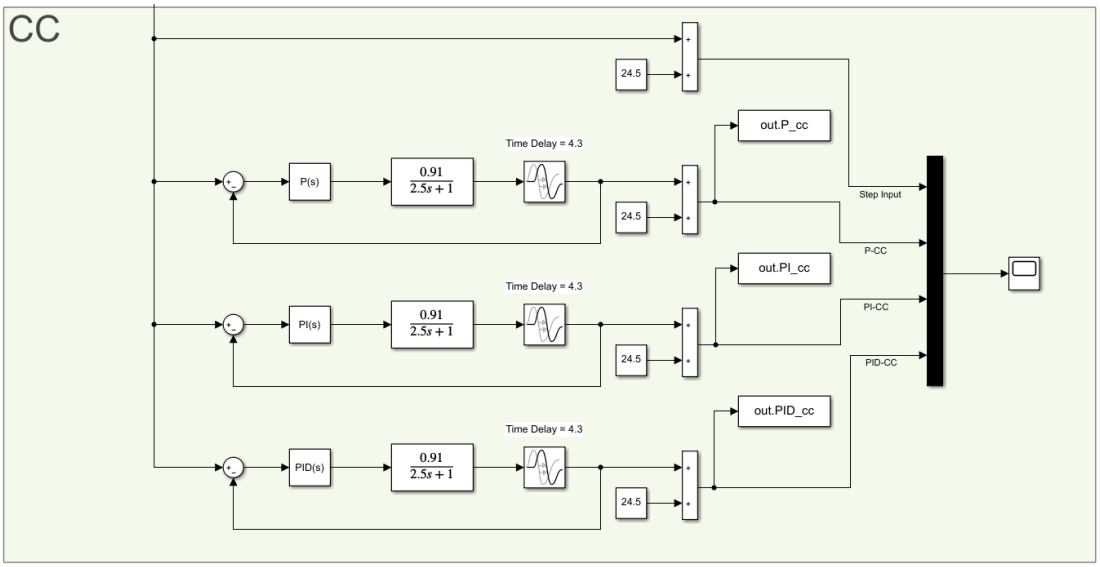
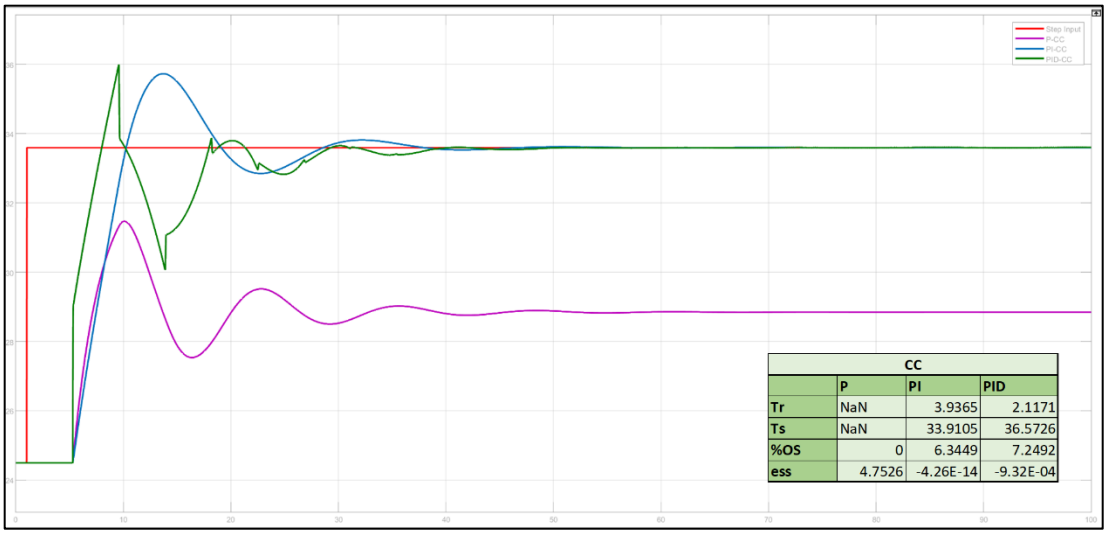


Figure 3.10: MATLAB Simulink diagram for CC (P, PI, PID) Step Test.



CC			
	P	PI	PID
Tr	NaN	3.9365	2.1171
Ts	NaN	33.9105	36.5726
%OS	0	6.3449	7.2492
ess	4.7526	-4.26E-14	-9.32E-04

Figure 3.11: Output Response & Dynamic Response for CC (P, PI, PID) Step Test.

DYNAMIC RESPONSE											
ZN				AMIGO			CC				
	P	PI	PID		PI	PID		P	PI	PID
Tr	Nan	73.3364	28.8412	Tr	225.5054	9.1060	Tr	NaN	3.9365	2.1171
Ts	Nan	143.0200	60.4721	Ts	405.6556	17.8792	Ts	NaN	33.9105	36.5726
%OS	0	0	0.0234	%OS	0	0.3596	%OS	0	6.3449	7.2492
ess	5.9513	1.44E-07	-0.0019	ess	0.0118	-4.97E-14	ess	4.7526	-4.26E-14	-9.32E-04

Figure 3.12: Summary of Dynamic Response for all ZN, AMIGO and CC.

Figure 3.7, Figure 3.9, Figure 3.11 above show the successful Step Test simulation result of the ZN, Amigo and CC Tuning rules respectively along with their Dynamic Response Table. It can be seen that P only tuning in both ZN and CC method results in a large e_{ss}, where the final process variable (PV) does not settle within ±2% of the set value (SV). Even though increasing K_p can reduce the e_{ss}, excessively large value of K_p may lead to increased overshoot, oscillation, and

instability, which is why K_p tends to be conservative. Implementation of K_i addresses this issue by accumulating the error over time and adjusting the control signal until the e_{ss} is eliminated. The introduction of K_d on the other hand causes the control signal to become sharper because it reacts to the rate of change of error. Sudden changes in the error can therefore produce large fluctuations in the control signal that makes it appear as jagged. This in results allows the system to improve its rise time (T_r) by providing a faster corrective response to rapid error changes.

In this plant, the PID-AMIGO tuning was able to achieve the control objective of minimum settling time due its well-balanced K_p, K_i, and K_d value. Its K_c is sufficiently high to provide a fast response with minimal overshoot and no sustained oscillations. K_i on the other hand effectively eliminates the e_{ss}, making it negligible as it is very close to 0. Finally, it has an appropriate K_d that rapidly reacts to the changes in the error, which directly increases T_r and accelerates the T_s of the system.

2. Setpoint Tracking

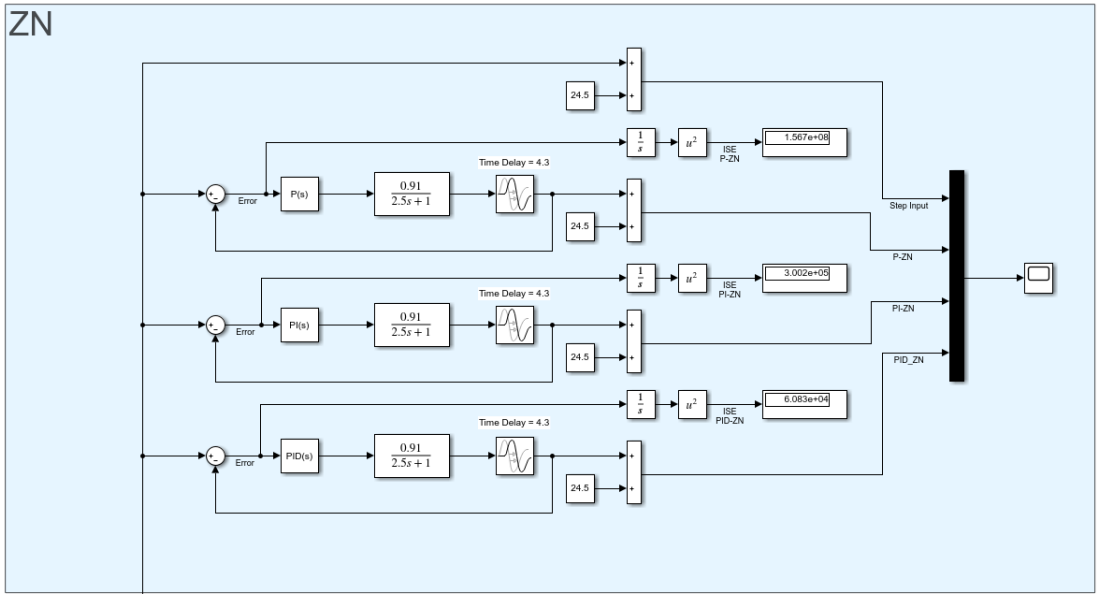


Figure 3.13: MATLAB Simulink diagram for ZN (P, PI, PID) Setpoint Tracking.

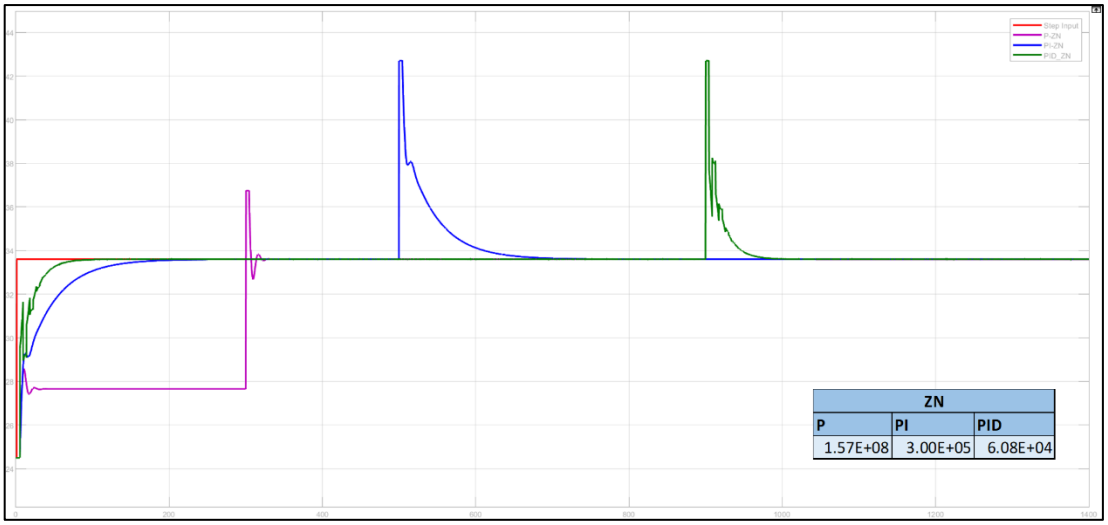


Figure 3.14: Output Response & Setpoint Tracking ISE for ZN (P, PI, PID).

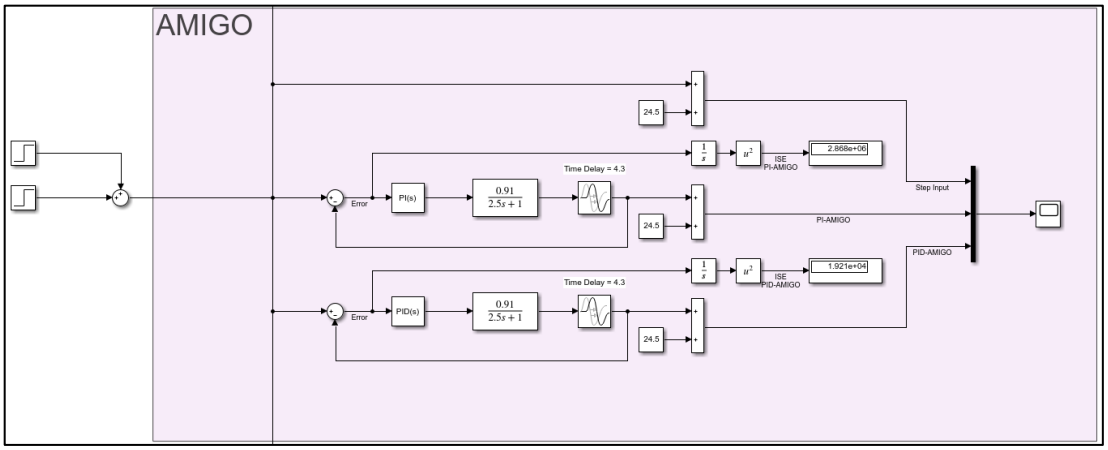


Figure 3.15: MATLAB Simulink diagram for AMIGO (PI, PID) Setpoint Tracking.

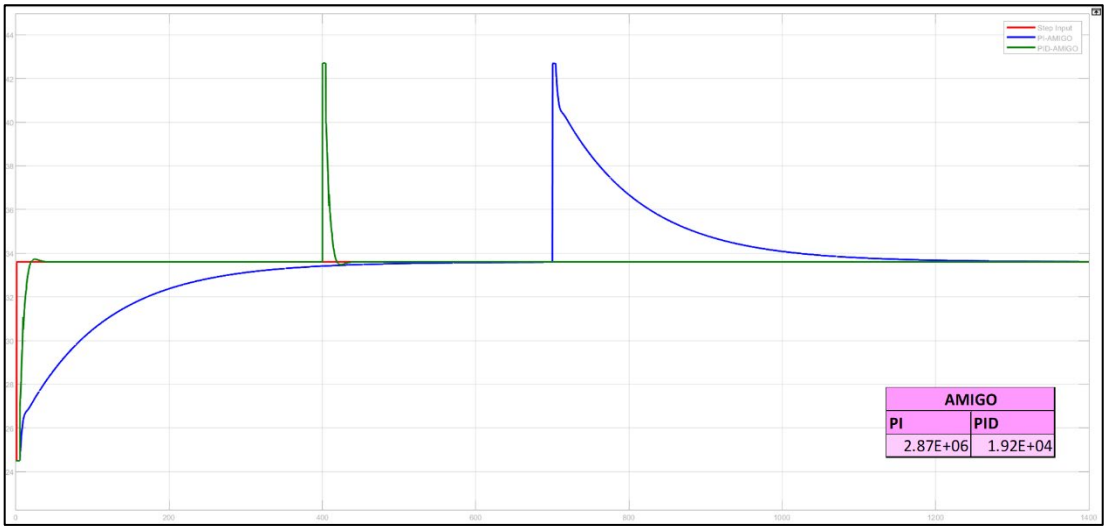


Figure 3.16: Output Response & Setpoint Tracking ISE for AMIGO (PI, PID).

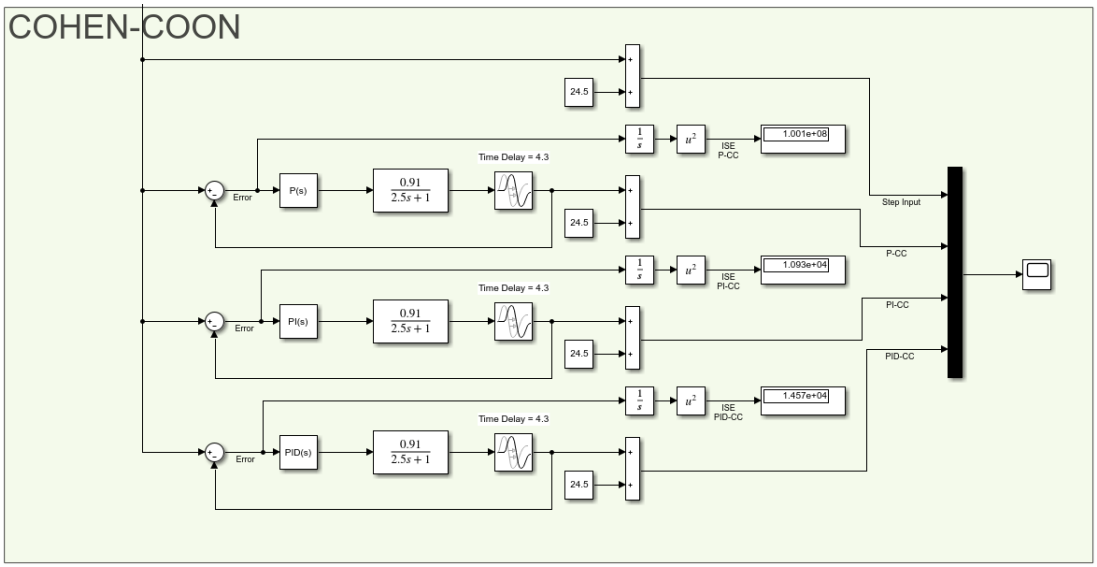


Figure 3.17: MATLAB Simulink diagram for CC (P, PI, PID) Setpoint Tracking.

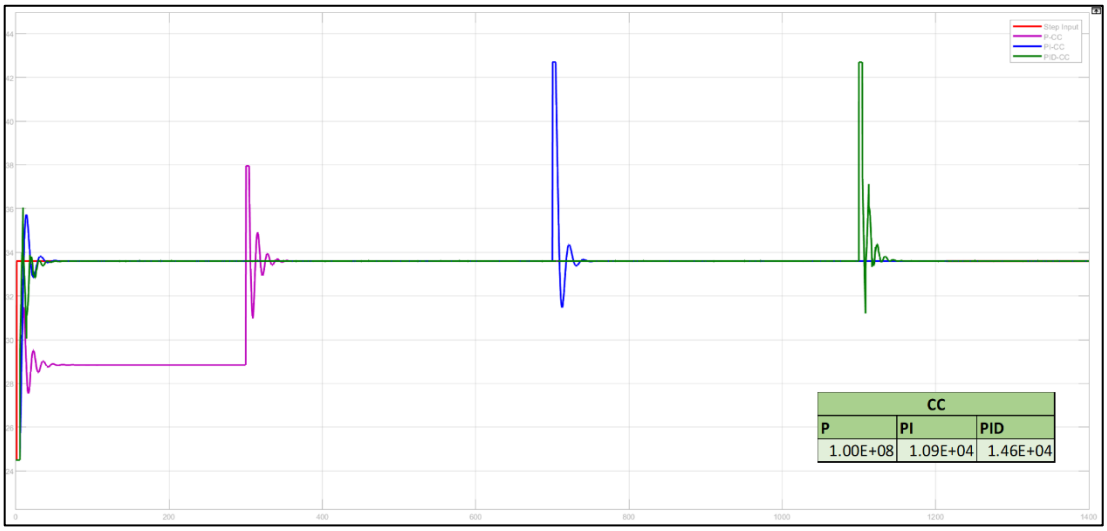


Figure 3.18: Output Response & Setpoint Tracking ISE for CC (P, PI, PID).

SETPOINT TRACKING - ISE									
ZN				AMIGO			CC		
P	PI	PID		PI	PID		P	PI	PID
1.57E+08	3.00E+05	6.08E+04		2.87E+06	1.92E+04		1.00E+08	1.09E+04	1.46E+04

Figure 3.19: Summary of Setpoint Tracking ISE for all ZN, AMIGO and CC.

Figure 3.14, Figure 3.16, Figure 3.18 shows the successful setpoint tracking performance for all levels of step input using the ZN, AMIGO, and CC PID tuning rules respectively along with their Setpoint Tracking ISE. All controllers display consistent behaviour and maintain their characteristic response across varying setpoints.

In this test, the PI-CC demonstrates the best performance in terms of having the smallest ISE. Even though PI-CC shows one the highest %OS, its moderately low Ki allows it to smoothly oscillates close to SV. As a result, it was accumulating only small errors within a long period of time, thus leading to a lower ISE but higher Ts compared to PID-AMIGO.

3. Disturbance Rejection

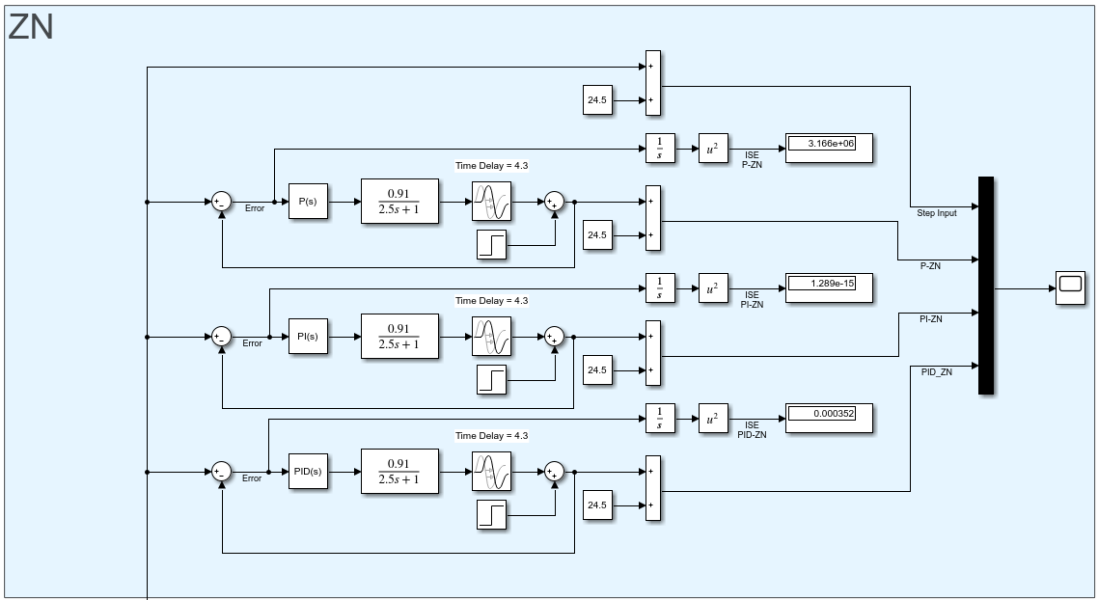


Figure 3.20: MATLAB Simulink diagram for ZN (P, PI, PID) Disturbance Rejection.

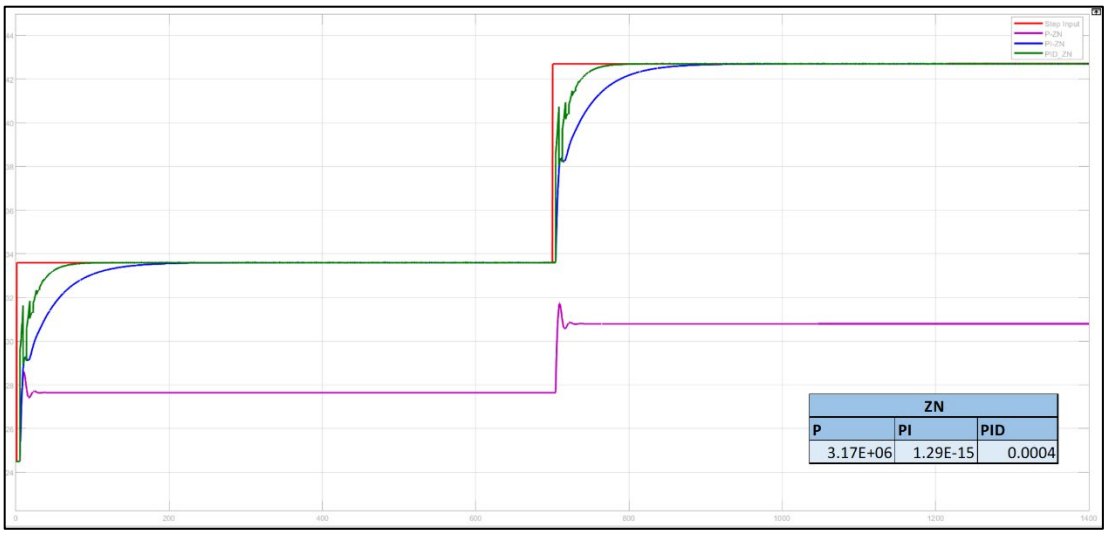


Figure 3.21: Output Response & Disturbance Rejection ISE for ZN (P, PI, PID).

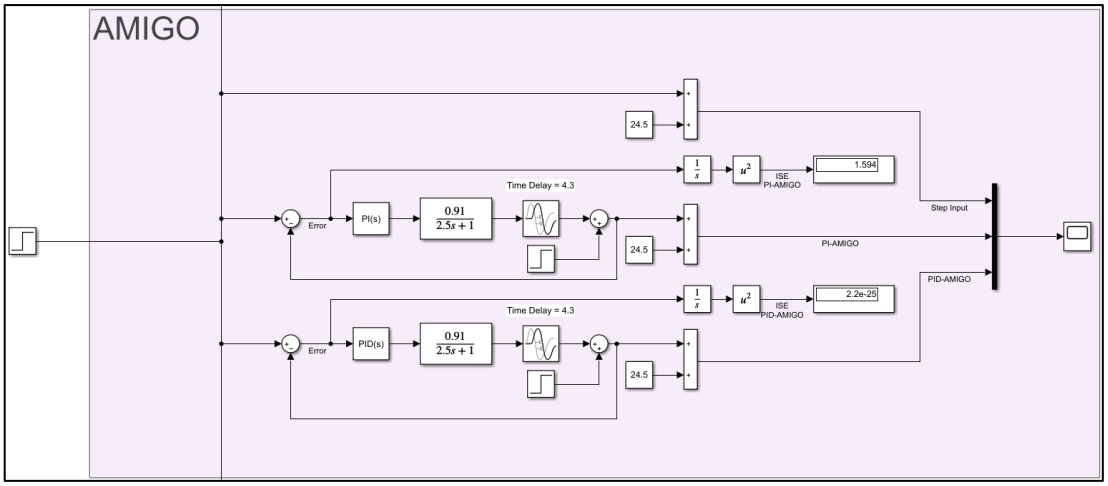


Figure 3.22: MATLAB Simulink diagram for AMIGO (PI, PID) Disturbance Rejection.

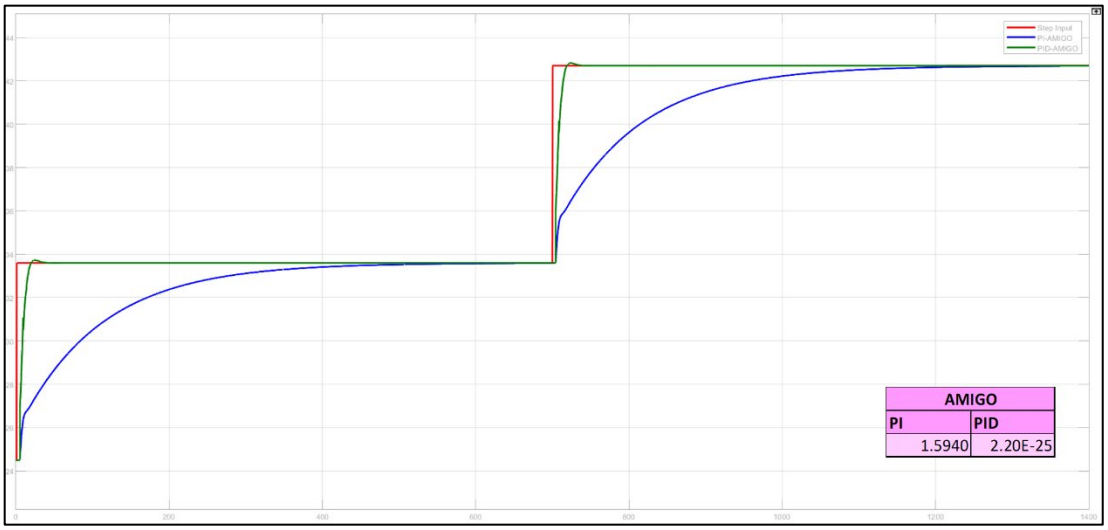


Figure 3.23: Output Response & Disturbance Rejection ISE for AMIGO (PI, PID).

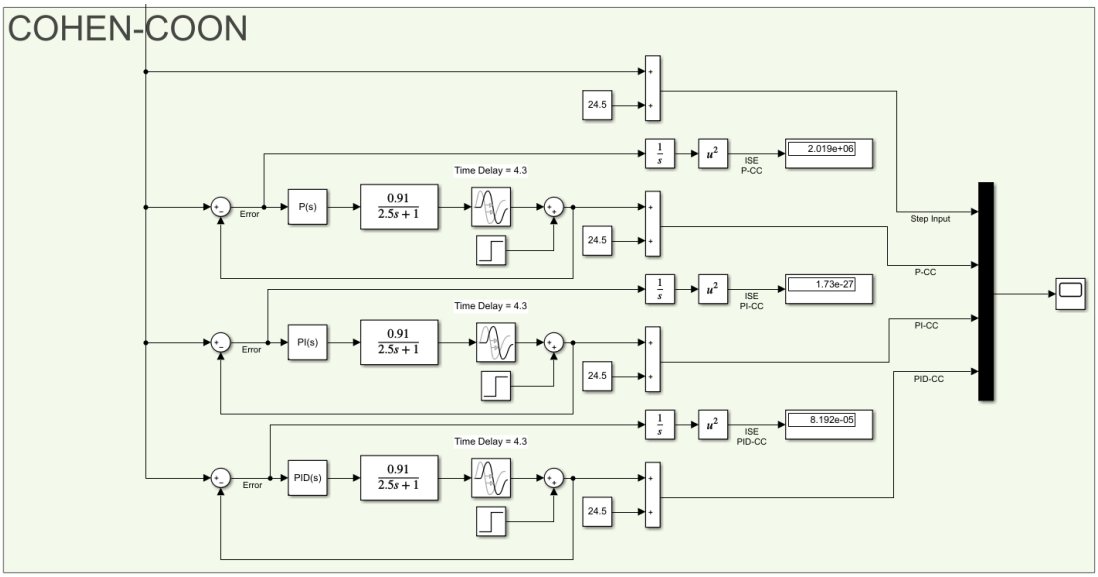


Figure 3.24: MATLAB Simulink diagram for CC (P, PI, PID) Disturbance Rejection.



Figure 3.25: Output Response & Disturbance Rejection ISE for CC (P, PI, PID).

DISTURBANCE REJECTION - ISE									
ZN				AMIGO			CC		
P	PI	PID		PI	PID		P	PI	PID
3.17E+06	1.29E-15	0.0004		1.5940	2.20E-25		2.02E+06	1.73E-27	8.19E-05

Figure 3.26: Summary of Disturbance Rejection ISE for all ZN, AMIGO and CC.

The disturbance rejection test demonstrates successful stabilization for all PID tuning methods. Each system was able to recover and stabilize despite the presence of disturbances, which were simulated by introducing a step input at random times. The disturbance response appears as an inverse or flipped version of the rising step response, as the controller acts to immediately counteract the deviation and bring the process variable back to the SV. Additionally, controllers that previously had large e_{ss} under P-only tuning (P-ZN and P-CC) were able to stabilize and reach the correct SV when a disturbance was introduced.

In this test, the PID-AMIGO performed the best, achieving the lowest ISE. The controller was able to immediately correct disturbances by aggressively adjusting the control signal to counteract the noise while maintaining a small undershoot and no sustained oscillations. As a result, the system returned to steady state very quickly, which shows fast disturbance rejection and stable behaviour.

Controller Selection

PID-AMIGO is selected to be applied in the flow control plant for having smallest T_s compared to other tuning methods, aligning with the control objective of minimum settling time. It also has the added benefit of smallest ISE in disturbance rejection test, and one of the best (3rd place) in setpoint tracking ISE performance.

3.3 Real plant PID response

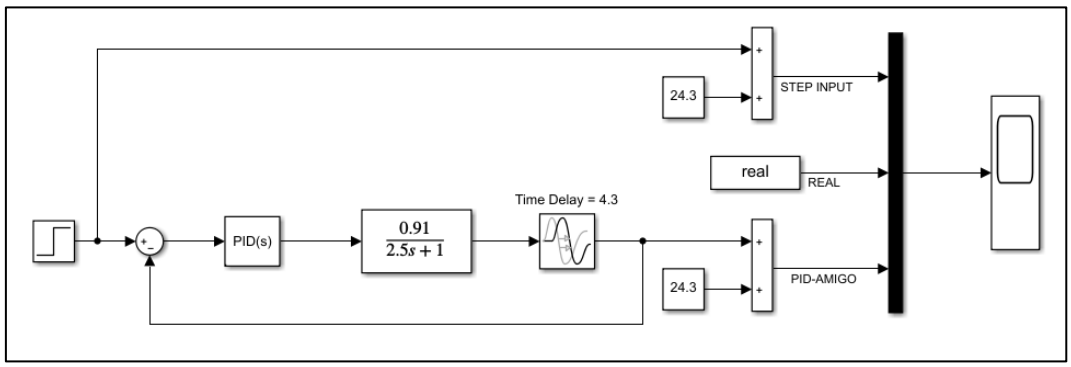


Figure 3.27: MATLAB Simulink diagram for Step Test of real plant PID response

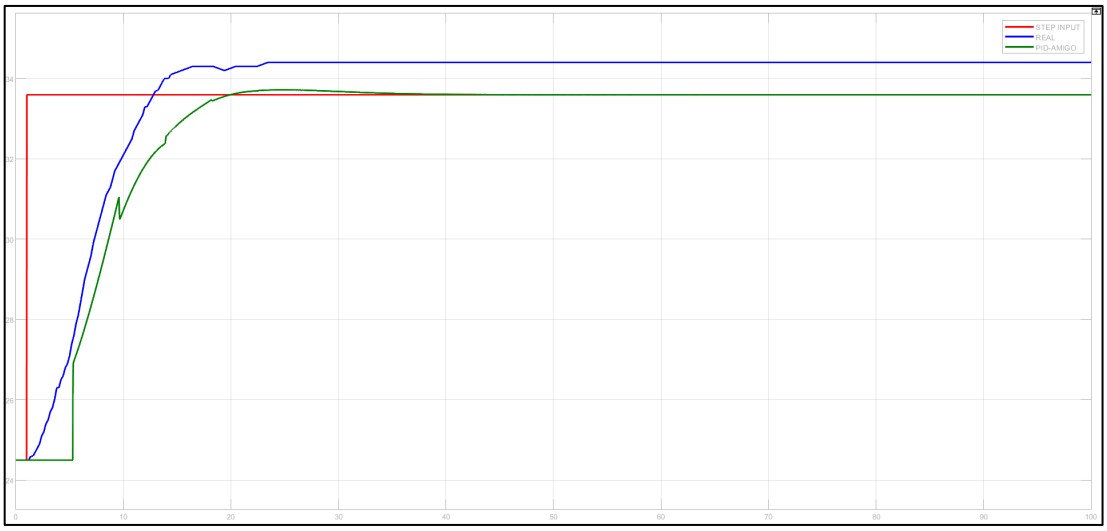


Figure 3.28: Output Response for Step Test of real plant PID response

```
real_dynamic =

  struct with fields:

       RiseTime: 8.1700
    SettlingTime: NaN
     SettlingMin: 32.7000
     SettlingMax: 34.4000
       Overshoot: 2.3810
      Undershoot: 0
            Peak: 34.4000
        PeakTime: 23.4000


ess_real =

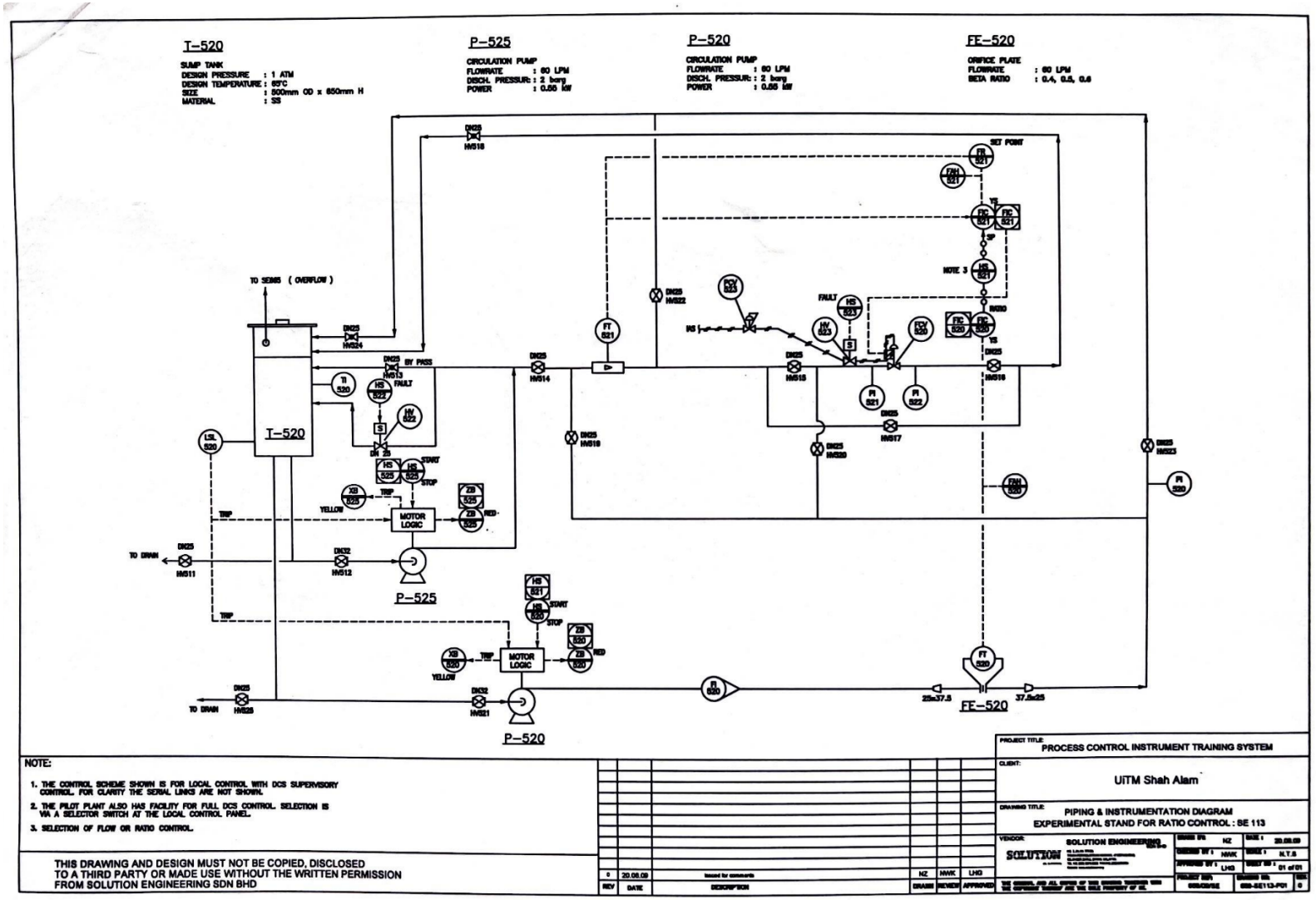
   -0.8000
```

Figure 3.29: Dynamic Response for real plant PID response

From the step test conducted on the real plant using the PID-AMIGO tuning method, it can be observed that the process behaviour is almost identical to the simulated response. The shape of the real plant response follows the simulation result closely which indicates that the PID-AMIGO controller is able to capture the dynamic response of the process accurately. This shows that the model used in simulation is reasonably representative of the actual plant behaviour.

However, a slight difference can be observed in the steady-state response. The settling value of the real plant is slightly higher than that of the simulated PID-AMIGO response, resulting in an e_{ss} of -0.800 that lies outside of the ±2% settling threshold. This discrepancy may be attributed to real hardware and mechanical factors such as unmodelled dynamics, sensor inaccuracies, or changes in plant conditions compared to the initial FOPDT data used for controller tuning. These effects are not fully captured in the simulation which leads to the observed deviation in the real plant response.

4.0 APPENDIX



Full P&I Diagram of the Flow Process Plant

Picture of Real Plant



Picture of Group Members



Python MSE Coding

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

# Simulation parameters
dt = 0.1  # Time step
t_end = 25
t = np.arange(0, t_end, dt)

# Input signal (unit step)
step_time = 1.0
step_value = 10.0

u = np.zeros_like(t)
u[t >= step_time] = step_value

# Original Transfer Function from excel:
data = np.loadtxt("rawdata.csv", delimiter=",", skiprows=1)
t_data = data[:, 1]
y_data = data[:, 3]

y_unknown = np.interp(t, t_data, y_data)
y_tf_delayed = y_unknown.copy()

# ==============================
# fopdt2 Method FOPDT
# ==============================
num_fopdt2 = [0.91]
den_fopdt2 = [2.5, 1]
sys_fopdt2 = signal.TransferFunction(num_fopdt2, den_fopdt2)
_, y_fopdt2, _ = signal.lsim(sys_fopdt2, U=u, T=t)

delay_fopdt2 = 4.3
delay_samples_fopdt2 = int(delay_fopdt2 / dt)

y_fopdt2_delayed = np.zeros_like(t)

if delay_samples_fopdt2 == 0:
    y_fopdt2_delayed = y_fopdt2.copy()
elif delay_samples_fopdt2 < len(t):
    y_fopdt2_delayed[delay_samples_fopdt2:] = y_fopdt2[:-delay_samples_fopdt2]

y_fopdt2_delayed += 24.5

# ==============================
# Sundaresan Method FOPDT
# ==============================
num_sundaresan = [0.91]
den_sundaresan = [1.943, 1]
sys_sundaresan = signal.TransferFunction(num_sundaresan, den_sundaresan)
_, y_sundaresan, _ = signal.lsim(sys_sundaresan, U=u, T=t)

delay_sundaresan = 11.335
delay_samples_sundaresan = int(delay_sundaresan / dt)

y_sundaresan_delayed = np.zeros_like(t)

if delay_samples_sundaresan == 0:
    y_sundaresan_delayed = y_sundaresan.copy()
elif delay_samples_sundaresan < len(t):
    y_sundaresan_delayed[delay_samples_sundaresan:] = y_sundaresan[:-delay_samples_sundaresan]

y_sundaresan_delayed += 24.5

# ==============================
# MSE calculation
# ==============================
error_fopdt2 = y_tf_delayed - y_fopdt2_delayed
error_sundaresan = y_tf_delayed - y_sundaresan_delayed

mse_fopdt2 = np.mean(error_fopdt2**2)
```

```python
mse_sundaresan = np.mean(error_sundaresan**2)

# ==============================
# Comparison
# ==============================
if mse_fopdt2 < mse_sundaresan:
    improvement = ((mse_sundaresan - mse_fopdt2) / mse_sundaresan) * 100
    comparison_text = (
        f'FOPDT 2 provides better\n'
        f'approximation (lower MSE)\n'
        f'{improvement:.2f}% better than Sundaresan Method'
    )
else:
    improvement = ((mse_fopdt2 - mse_sundaresan) / mse_fopdt2) * 100
    comparison_text = (
        f'SUNDARESAN METHOD provides better\n'
        f'approximation (lower MSE)\n'
        f'{improvement:.2f}% better than FOPDT 2'
    )

# ==============================
# Parameters
# ==============================
K_fopdt2 = num_fopdt2[0]
tau_fopdt2 = den_fopdt2[0]
theta_fopdt2 = delay_fopdt2

K_sundaresan = num_sundaresan[0]
tau_sundaresan = den_sundaresan[0]
theta_sundaresan = delay_sundaresan

# ==============================
# Plot
# ==============================
plt.figure(figsize=(12, 6))
```

```python
plt.plot(t, y_tf_delayed, 'b-', linewidth=2, label='Unknown Transfer Function')
plt.plot(t, y_fopdt2_delayed, 'r--', linewidth=2,
         label=f'FOPDT 2 Method (MSE: {mse_fopdt2:.6f})')
plt.plot(t, y_sundaresan_delayed, 'g-.', linewidth=2,
         label=f'Sundaresan Method (MSE: {mse_sundaresan:.6f})')

textstr = (
    'FOPDT Models:\n\n'
    'FOPDT 2 Method:\n'
    rf'$H_2(s) = \frac{{{K_fopdt2}}}{{{tau_fopdt2}s + 1}} e^{{-{theta_fopdt2}s}}$'
    '\n\nSundaresan Method:\n'
    rf'$H_1(s) = \frac{{{K_sundaresan}}}{{{tau_sundaresan}s + 1}} e^{{-{theta_sundaresan}s}}$'
    '\n\n————————————————————\n'
    + comparison_text
)

props = dict(boxstyle='round', facecolor='white',
             edgecolor='red', linewidth=2, alpha=0.9)

plt.text(0.98, 0.35, textstr, transform=plt.gca().transAxes,
         fontsize=11, verticalalignment='center',
         horizontalalignment='right', bbox=props)

plt.xlabel('Time (s)')
plt.ylabel('Output')
plt.title('FOPDT Comparison - GROUP 5', fontweight='bold')
plt.legend(loc='upper left')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

MATLAB Dynamic Response script

```
% DYNAMIC RESPONSE
settling_value = 33.6;
znP     = stepinfo(out.P_zn.Data, out.P_zn.Time, settling_value);
znPI    = stepinfo(out.PI_zn.Data, out.PI_zn.Time, settling_value);
znPID   = stepinfo(out.PID_zn.Data, out.PID_zn.Time, settling_value);

amigoPI    = stepinfo(out.PI_amigo.Data, out.PI_amigo.Time, settling_value);
amigoPID   = stepinfo(out.PID_amigo.Data, out.PID_amigo.Time, settling_value);

ccP     = stepinfo(out.P_cc.Data, out.P_cc.Time, settling_value);
ccPI    = stepinfo(out.PI_cc.Data, out.PI_cc.Time, settling_value);
ccPID   = stepinfo(out.PID_cc.Data, out.PID_cc.Time, settling_value);

% ESS
ess_znP     = settling_value-out.P_zn.Data(end)
ess_znPI    = settling_value-out.PI_zn.Data(end)
ess_znPID   = settling_value-out.PID_zn.Data(end)

ess_amigoPI     = settling_value-out.PI_amigo.Data(end)
ess_amigoPID    = settling_value-out.PID_amigo.Data(end)

ess_ccP     = settling_value-out.P_cc.Data(end)
ess_ccPI    = settling_value-out.PI_cc.Data(end)
ess_ccPID   = settling_value-out.PID_cc.Data(end)

```